Model components for fitted models with plm

Yves Croissant

2025-11-13

plm tries to follow as close as possible the way models are fitted using `lm`. This relies on the following steps, using the `formula-data` with some modifications:

- compute internally the `model.frame` by getting the relevant arguments (`formula`, `data`, `subset`, `weights`, `na.action` and `offset`) and the supplementary argument,
- extract from the `model.frame` the response `y` (with `pmodel.response`) and the model matrix `X` (with `model.matrix`),
- call the (non-exported) estimation function `plm.fit` with `X` and `y` as arguments.

Panel data has a special structure which is described by an `index` argument. This argument can be used in the `pdata.frame` function which returns a `pdata.frame` object. A `pdata.frame` can be used as input to the `data` argument of `plm`. If the `data` argument of `plm` is an ordinary `data.frame`, the `index` argument can also be supplied as an argument of `plm`. In this case, the `pdata.frame` function is called internally to transform the data.

Next, the `formula`, which is the first and mandatory argument of `plm` is coerced to a `Formula` object.

`model.frame` is then called, but with the `data` argument in the first position (a `pdata.frame` object) and the `formula` in the second position. This unusual order of the arguments enables to use a specific `model.frame.pdata.frame` method defined in `plm`.

As for the `model.frame.formula` method, a `data.frame` is returned, with a `terms` attribute.

Next, the `X` matrix is extracted using `model.matrix`. The usual way to do so is to feed the function with two arguments, a `formula` or a `terms` object and a `data.frame` created with `model.frame`. `lm` uses something like `model.matrix(terms(mf), mf)` where `mf` is a `data.frame` created with `model.frame`. Therefore, `model.matrix` needs actually one argument and not two and we therefore wrote a `model.matrix.pdata.frame` which does the job ; the method first checks that the argument has a `term` attribute, extracts the `terms` (actually the `formula`) and then computes the model's matrix `X`.

The response `y` is usually extracted using `model.response`, with a `data.frame` created with `model.frame` as first argument, but it is not generic. We therefore created a generic called `pmodel.response` and provide a `pmodel.response.pdata.frame` method. We illustrate these features using a simplified (in terms of covariates) example with the `SeatBelt` data set:

```
library("plm")
data("SeatBelt", package = "pder")
SeatBelt$occfat <- with(SeatBelt, log(farsocc / (vmtrural + vmturban)))
pSB <- pdata.frame(SeatBelt)
```

We start with an OLS (pooling) specification:

```
formols <- occfat ~ log(usage) + log(percapin)
mfols <- model.frame(pSB, formols)
Xols <- model.matrix(mfols)
y <- pmodel.response(mfols)
coef(lm.fit(Xols, y))
```

```
##   (Intercept)    log(usage) log(percapin)
##     7.4193570     0.1657293    -1.1583712
```

which is equivalent to:

```
coef(plm(formols, SeatBelt, model = "pooling"))
```

```
##   (Intercept)    log(usage) log(percapin)
##     7.4193570     0.1657293    -1.1583712
```

Next, we use an instrumental variables specification. Variable `usage` is endogenous and instrumented by three variables indicating the law context: `ds`, `dp`, and `dsp`.

The model is described using a two-parts formula, the first part of the RHS describing the covariates and the second part the instruments. The following two formulations can be used:

```
formiv1 <- occfat ~ log(usage) + log(percapin) | log(percapin) + ds + dp + dsp
formiv2 <- occfat ~ log(usage) + log(percapin) | . - log(usage) + ds + dp + dsp
```

The second formulation has two advantages:

- in the common case when a lot of covariates are instruments, these covariates don't need to be indicated in the second RHS part of the formula,
- the endogenous variables clearly appear as they are proceeded by a `-` sign in the second RHS part of the formula.

The formula is coerced to a `Formula`, using the `Formula` package. `model.matrix.pdata.frame` then internally calls `model.matrix.Formula` in order to extract the covariates and instruments model matrices:

```
mfSB1 <- model.frame(pSB, formiv1)
X1 <- model.matrix(mfSB1, rhs = 1)
W1 <- model.matrix(mfSB1, rhs = 2)
head(X1, 3) ; head(W1, 3)
```

```
##    (Intercept) log(usage) log(percapin)
## 8            1 -0.7985077      9.955748
## 9            1 -0.4155154      9.975622
## 10           1 -0.4155154     10.002110
```

```
##    (Intercept) log(percapin) ds dp dsp
## 8            1      9.955748  0  0   0
## 9            1      9.975622  1  0   0
## 10           1     10.002110  1  0   0
```

For the second (and preferred formulation), the `dot` argument should be set and is passed to the `Formula` methods. `.` has actually two meanings:

- all available covariates,
- the previous covariates used while updating a formula.

which correspond respectively to `dot = "seperate"` (the default) and `dot = "previous"`. See the difference between the following two examples:

```
library("Formula")
head(model.frame(Formula(formiv2), SeatBelt), 3)
```

```
##       occfat log(usage) log(percapin) state year farsocc farsnocc usage
## 8  -3.788976 -0.7985077      9.955748    AK 1990      90        8  0.45
## 9  -3.904837 -0.4155154      9.975622    AK 1991      81       20  0.66
## 10 -3.699611 -0.4155154     10.002110    AK 1992      95       13  0.66
##    percapin unemp  meanage   precentb   precenth  densurb   densrur
## 8     21073  7.05 29.58628 0.04157167 0.03252657 1.099419 0.1906836
## 9     21496  8.75 29.82771 0.04077293 0.03280357 1.114670 0.1906712
## 10    22073  9.24 30.21070 0.04192957 0.03331731 1.114078 0.1672785
##         viopcap    proppcap vmtrural vmturban fueltax lim65 lim70p mlda21 bac08
## 8  0.0009482704 0.008367458     2276     1703       8     0      0      1     0
## 9  0.0010787370 0.008940661     2281     1740       8     0      0      1     0
## 10 0.0011257068 0.008366873     2005     1836       8     1      0      1     0
##    ds dp dsp
## 8   0  0   0
## 9   1  0   0
## 10  1  0   0
```

```
head(model.frame(Formula(formiv2), SeatBelt, dot = "previous"), 3)
```

```
##       occfat log(usage) log(percapin) ds dp dsp
## 8  -3.788976 -0.7985077      9.955748  0  0   0
## 9  -3.904837 -0.4155154      9.975622  1  0   0
## 10 -3.699611 -0.4155154     10.002110  1  0   0
```

In the first case, all the covariates are returned by `model.frame` as the `.` is understood by default as "everything".

In `plm`, the `dot` argument is internally set to `previous` so that the end-user doesn't have to worry about these subtleties.

```
mfSB2 <- model.frame(pSB, formiv2)
X2 <- model.matrix(mfSB2, rhs = 1)
W2 <- model.matrix(mfSB2, rhs = 2)
head(X2, 3) ; head(W2, 3)
```

```
##    (Intercept) log(usage) log(percapin)
## 8            1 -0.7985077      9.955748
## 9            1 -0.4155154      9.975622
## 10           1 -0.4155154     10.002110
```

```
##    (Intercept) log(percapin) ds dp dsp
## 8            1      9.955748  0  0   0
## 9            1      9.975622  1  0   0
## 10           1     10.002110  1  0   0
```

The IV estimator can then be obtained as a 2SLS estimator: First, regress the covariates on the instruments and get the fitted values:

```
HX1 <- lm.fit(W1, X1)$fitted.values
head(HX1, 3)
```

```
##    (Intercept) log(usage) log(percapin)
## 8            1 -1.0224257      9.955748
## 9            1 -0.5435055      9.975622
## 10           1 -0.5213364     10.002110
```

Next, regress the response on these fitted values:

```
coef(lm.fit(HX1, y))
```

```
##   (Intercept)    log(usage) log(percapin)
##     7.5641209     0.1768576    -1.1722590
```

The same can be achieved in one command by using the `formula-data` interface with `plm`:

```
coef(plm(formiv1, SeatBelt, model = "pooling"))
```

```
##   (Intercept)    log(usage) log(percapin)
##     7.5641209     0.1768576    -1.1722590
```

or with the `ivreg` function from package `AER` (or with the newer function `ivreg` in package `ivreg` superseding `AER::ivreg()`):

```
coef(AER::ivreg(formiv1, data = SeatBelt))
```

```
##   (Intercept)    log(usage) log(percapin)
##     7.5641209     0.1768576    -1.1722590
```